Chapitre 3 : Les tableaux

Hassen NAKBI ::: hassen.nakbi@gmail.com

1ere année Licence

16 octobre 2023

Introduction

Pourquoi les tableaux ?

Une variable permet de mémoriser une seule information de type simple. Cependant, si nous devons traiter un ensemble de valeurs homogènes, il serait nécessaire de déclarer et d'utiliser plusieurs variables pour conserver ces informations en mémoire.
Par conséquent, cela peut rendre le traitement plus complexe en raison de la nécessité de répéter des actions communes pour chaque variable et de distinguer les différents noms qui sont associés aux variables.

Introduction

Les tableaux

Les tableaux sont conçus pour stocker et manipuler des ensembles de données de manière structurée et efficace. Ils sont utilisés pour gérer des collections de valeurs homogènes en les organisant dans une structure de données cohérente.

2 Notion de tableau

- Définition
- Propriétés
- Opérations

Définition

- **Un tableau est une structure de données** qui permet de regrouper un ensemble des valeurs de même type.
- En algorithmique, un **tableau est un nouveau de type de donnée** complexe homogène, qui peut accepter plusieurs informations.

2 Notion de tableau

- Définition
- Propriétés
- Opérations

Propriétés

- Un tableau est représenté en mémoire de façon contiguë, tous les éléments constituant le tableau sont placés en mémoire les uns après les autres.
- Un tableau qui représente un seul ensemble des valeurs est un **tableau unidimensionnel**, s'il possède plusieurs ensembles est un **tableau bidimensionnel**.
- La **taille** d'un tableau est le nombre d'éléments que le tableau peut contenir au plus, il est représenté par **une valeur constante**.
- Un **élément du tableau** désigne une seule valeur du tableau.
- Un **indice** d'un élément est le numéro de position d'une valeur par rapport au début du tableau.

2 Notion de tableau

- Définition
- Propriétés
- Opérations

Opérations

Dans un tableau, plusieurs opérations essentielles sont couramment effectuées pour manipuler et gérer les données.

- Le chargement du tableau ou le remplissage d'un ensemble de valeurs peut se faire de trois manières : depuis le clavier, de manière aléatoire ou à partir d'une liste initiale fournie par le programmeur.
- L'affichage de la liste des valeurs du tableau à l'écran.
- La recherche d'existence d'une valeur spécifique, ou bien les occurrences d'une valeur dans un tableau.
- L'opération de tri consiste à réorganiser les éléments du tableau dans un ordre spécifique, tel que l'ordre croissant ou décroissant.

3 Tableau unidimensionnel

- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage
- L'opération de tri
- L'opération de recherche

Syntaxe de définition d'un tableau

Algorithmique

```
CONST N = valeur
TYPE TAB=tableau[1 .. N] de type
VAR T : TAB
DÉBUT
FIN
```

Langage C

```
#define N valeur
type T[N];
int main( ) {
return 0;}
```

Exemple en Algorithmique

```
CONST N = 50
TYPE TAB=tableau[1 .. N] d'entiers
TYPE LIST=tableau[1 .. N] de réels
VAR T : TAB
      L : LIST
DÉBUT
FIN
```

Exemple Langage C

```
#define N 50
short T[N];
float L[N];
int main( ) {
return 0;}
```

3 Tableau unidimensionnel

- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage
- L'opération de tri
- L'opération de recherche

Remplissage au clavier en Algorithmique

```
CONST N = 50
TYPE TAB=tableau[1 .. N] d'entiers
VAR T : TAB
      i : entier
DÉBUT
Pour i de 1 à N pas←—1 faire
début
Écrire("T[",i,"]=")
Lire(T[i])
FinPour
FIN
```

Remplissage au clavier en Langage C

```
#define N 50
short T[N],i;
int main( ) {
for(i=0;i< N;i++){
printf("T[%hi]=",i);
scanf("%hi",&T[i]);}
return 0;}
```

Remplissage aléatoire en Algorithmique

```
CONST N = 50
TYPE TAB=tableau[1 .. N] d'entiers
VAR T : TAB
      i : entier
DÉBUT
Pour i de 1 à N pas⟵1 faire
T[i] ⟵ -7 + aléa(11)
FinPour
FIN
```

Remplissage aléatoire en Langage C

```
#include <stdlib.h>
#define N 50
short T[N],i;
int main( ) {
for(i=0;i < N;i++){
T[i] = -7 + rand()%11;}
return 0;}
```

Remplissage par liste initiale en Langage C

```
#define N 50
short T[N]={11,-3,5,23,7};
int main( ) {
return 0;}
```

3 Tableau unidimensionnel

- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage
- L'opération de tri
- L'opération de recherche

L'affichage en Algorithmique

```
CONST N = 50
TYPE TAB=tableau[1 .. N] d'entiers
VAR T : TAB
      i : entier
DÉBUT
Pour i de 1 à N pas⟵1 faire
Écrire(T[i],",")
FinPour
FIN
```

L'affichage en Langage C

```
#include <stdio.h>
#define N 50
short T[N],i ;
int main( ) {
for(i=0 ;i < N ;i++){
printf("%hi, ",T[i]) ;}
return 0 ;}
```

3 Tableau unidimensionnel

- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage
- L'opération de tri
- L'opération de recherche

Trier un tableau

L'opération de tri consiste à réorganiser un ensemble d'éléments dans un ordre particulier, généralement croissant ou décroissant, en fonction de la valeur de ces éléments. L'objectif du tri est de rendre les données plus faciles à rechercher, à afficher ou à traiter.
Les algorithmes de tri les plus fréquemment utilisés sont le tri par insertion, le tri à bulles, le tri par sélection.

Tri à bulles

Principe de tri à bulles

Le tri à bulles est un algorithme de tri simple mais inefficace. Il tire son nom du fait qu'il "fait remonter" les éléments non triés comme des "bulles" à travers le tableau jusqu'à ce qu'ils atteignent leur position correcte.
Son principe dans chaque itération du tableau, on compare deux éléments successifs, puis on effectue un échange entre les éléments qui ne sont pas dans l'ordre, jusqu'à ce qu'une itération se termine sans échange. À la fin du premier parcours du tableau, la valeur maximale ou minimale est placée dans la position correcte.

Exemple de tri à bulles



Algorithme Tri à bulles

```
CONST N = 50
TYPE TAB=tableau[1 .. N]
d'entiers
VAR T : TAB
      i,S,ax : entier
      xchg : booléen
DÉBUT
S ⟵ N - 1
Répéter
xchg ⟵ faux
Pour i de 1 à S pas⟵1 faire
début
SI (T[i] > T[i+1]) alors
ax ⟵ T[i]
T[i] ⟵ T[i+1]
T[i+1] ⟵ ax
xchg ⟵ vrai
FinSI
FinPour
S ⟵ S - 1
Jusqu'à (xchg = faux)
FIN
```

Tri par sélection

Principe de tri par sélection

Le tri par sélection est un algorithme de tri simple, mais il est plus efficace que le tri à bulles.
Il divise le tableau en deux parties : une partie triée et une partie non triée. Au début, la partie triée est considérée le premier élément, tandis que la partie non triée est le reste du tableau.
À chaque itération, l'algorithme parcourt la partie non triée pour trouver l'élément le plus petit ou le plus grand et le place par échange à la fin de la partie triée.
L'algorithme continue de cette manière jusqu'à ce que la partie non triée soit vide, ce qui signifie que le tableau entier est trié.

Exemple de tri par sélection

1er parcours
échange entre 11 et 3

11	5	9	3	7

2ème parcours
pas d'échange

3	5	9	11	7

3ème parcours
échange entre 9 et 7

3	5	9	11	7

4ème parcours
échange entre 11 et 9

3	5	7	11	9

3	5	7	9	11

Algorithme par sélection

```
CONST N = 50
TYPE TAB=tableau[1 .. N]
d'entiers
VAR T : TAB
      i,j,m,ax : entier
DÉBUT
Pour i de 1 à N-1 pas←—1 faire
début
m ⟵ i
Pour j de i+1 à N pas←—1
faire
début
SI (T[j] > T[m]) alors
m ⟵ j
FinSI
FinPour
SI (i ≠ m) alors
ax ⟵ T[i]
T[i] ⟵ T[m]
T[m] ⟵ ax
FinSI
FinPour
FIN
```

Tri par insertion

Principe de tri par insertion

Le tri par insertion est un algorithme efficace pour les petites quantités de données.
L'algorithme considère initialement que le premier élément du tableau est trié. Il parcourt ensuite le reste du tableau, un élément à la fois, et à chaque étape, il insère cet élément dans la bonne position parmi les éléments précédemment triés.
Pour insérer un élément dans la partie triée, il compare aux éléments déjà triés, en déplaçant les éléments plus grands ou plus petits vers la droite pour faire de la place, jusqu'à ce qu'il trouve sa position appropriée.

Exemple de tri par insertion



Algorithme par insertion

```
CONST N = 50
TYPE TAB=tableau[1 .. N]
d'entiers
VAR T : TAB
      i,j,vi : entier
DÉBUT
Pour i de 2 à N pas←1 faire
début
j ⟵ i-1
vi ⟵ T[i]
Tant que(T[j] > vi ET j > 0)
faire
T[j+1] ⟵ T[j]
j ⟵ j-1
FinTant que
T[j+1] ⟵ vi
FinPour
FIN
```

3 Tableau unidimensionnel

- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage
- L'opération de tri
- L'opération de recherche

Recherche séquentielle

La recherche séquentielle est une méthode simple de recherche d'un élément spécifique dans un tableau de données. Le processus implique de parcourir séquentiellement chaque élément de la liste, un par un, jusqu'à ce que l'élément recherché soit trouvé ou que la fin de la liste soit atteinte.

Algorithme par recherche séquentielle

```
CONST N = 50
TYPE TAB=tableau[1 .. N] d'entiers
VAR T : TAB
      i,val : entier
DÉBUT
Lire(val)
i ⟵ 1
Tant que(T[i] ≠ val ET i <= N) faire
i ⟵ i+1
FinTant que
SI (T[i] = val) alors
Écrire("Élément trouvé dans la position ",i)
SINON
Écrire("Aucun élément trouvé")
FinSI
FIN
```

Recherche dichotomique

La recherche dichotomique est une méthode efficace pour trouver la position d'un élément spécifique dans un tableau trié.
L'algorithme commence par comparer l'élément recherché avec l'élément milieu du tableau. S'il est égal au milieu, la recherche est terminée avec succès.
Si l'élément recherché est plus petit que le milieu, la recherche se poursuit sur la moitié gauche du tableau, sinon sur la moitié droite du tableau.
Le cycle se répète avec la moitié restante du tableau, en divisant la partie restante en deux, jusqu'à ce que l'élément recherché soit trouvé ou que tableau devient vide.

Exemple de recherche dichotomique



Algorithme par recherche dichotomique

```
CONST N = 50
TYPE TAB=tableau[1 .. N]
d'entiers
VAR T : TAB
      d,f,m,val : entier
DÉBUT
Lire(val)
d ⟵ 1
f ⟵ N
Répéter
m ⟵ (d + f) DIV 2
SI (T[m] > val) alors
f ⟵ m - 1
FinSI
SI (T[(m] < val) alors
d ⟵ m + 1
FinSI
Jusqu'à(T[m]=val OU d > f)
SI (T[m] = val) alors
Écrire("élément trouvé dans la
position ",m)
SINON
Écrire("Aucun élément")
FinSI
FIN
```

4 Tableau à deux dimensions

- Définition
- Propriétés
- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage

Tableau à deux dimensions

Définition

Un tableau à deux dimensions également appelé matrice est une structure de données rectangulaire composée de lignes et de colonnes, ou chaque élément individuel situé à un emplacement précis dans la matrice. Il représente la valeur conservée à l'intersection de couple ligne-colonne.

Exemple

Tableau bidimensionnel : Matrice

	Colonne 1	Colonne 2	Colonne 3
Ligne 1			**Cellule 1,3**
Ligne 2			
Ligne 3		**Cellule 3,2**	

4 Tableau à deux dimensions

- Définition
- Propriétés
- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage

Propriétés

- Un tableau à deux dimensions est une structure rectangulaire composée de lignes et de colonnes.
- La taille d'un tableau à deux dimensions est le produit de son nombre de lignes et de son nombre de colonnes, m x n.
- Tous les éléments d'un tableau à deux dimensions sont du même type de données, des valeurs homogènes.
- Les éléments d'un tableau à deux dimensions sont les valeurs stockées à l'intersection des lignes et des colonnes. Chaque élément est identifié par ses coordonnées (i, j), où "i" est le numéro de ligne et "j" est le numéro de colonne.

4 Tableau à deux dimensions

- Définition
- Propriétés
- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage

Syntaxe de définition d'un tableau

Algorithmique

```
CONST M=valeur, N=valeur
TYPE MAT=tableau[1 .. M][1 .. N] de type
VAR M : MAT
DÉBUT
FIN
```

Langage C

```
#define M valeur
#define N valeur
type M[M][N];
int main( ) {
return 0;}
```

Exemple en Algorithmique

```
CONST M=20, N=30
TYPE MAT=tableau[1 .. M][1 .. N] d'entiers
TYPE LST=tableau[1 .. M][1 .. N] de réels
VAR MT : MAT
     L : LST
DÉBUT
FIN
```

Exemple Langage C

```
#define M 20
#define N 30
short MT[M][N];
float L[M][N];
int main( ) {
return 0;}
```

4 Tableau à deux dimensions

- Définition
- Propriétés
- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage

Remplissage au clavier en Algorithmique

```
CONST M=20, N=30
TYPE MAT=tableau[1 .. M][1 .. N] d'entiers
VAR MT : MAT
      i,j : entier
DÉBUT
Pour i de 1 à M pas⟵1 faire
Pour j de 1 à N pas⟵1 faire
début
Écrire("MT[",i,j,"]=")
Lire(MT[i][j])
FinPour
FIN
```

Remplissage au clavier en Langage C

```
#include <stdio.h>
#define M 20
#define N 30
short MT[M][N],i,j ;
int main( ) {
for(i=0 ;i< M ;i++){
for(j=0 ;j< N ;j++){
printf("MT[%hi,%hi]=",i,j) ;
scanf("%hi",&MT[i][j]) ;}}
return 0 ;}
```

Remplissage aléatoire en Algorithmique

```
CONST M=20, N = 30
TYPE MAT=tableau[1 .. M][1 .. N] d'entiers
VAR MT : MAT
      i,j : entier
DÉBUT
Pour i de 1 à M pas⟵1 faire
Pour j de 1 à N pas⟵1 faire
MT[i][j] ⟵ -7 + aléa(11)
FinPour
FIN
```

Remplissage aléatoire en Langage C

```
#include <stdlib.h>
#define M 20
#define N 30
short MT[M][N],i,j ;
int main( ) {
for(i=0 ;i < M ;i++){
for(j=0 ;j < N ;j++){
MT[i][j] = -7 + rand()%11 ;}}
return 0 ;}
```

4 Tableau à deux dimensions

- Définition
- Propriétés
- Syntaxe de définition
- L'opération de remplissage
- L'opération de l'affichage

L'affichage en Algorithmique

```
CONST M=20, N = 30
TYPE MAT=tableau[1 .. M][1 .. N] d'entiers
VAR MT : MAT
      i,j : entier
DÉBUT
Pour i de 1 à M pas⟵1 faire
Pour j de 1 à N pas⟵1 faire
Écrire(MT[i][j],",")
FinPour
FIN
```

L'affichage en Langage C

```
#include <stdio.h>
#define N 20
#define M 30
short MT[M][N],i,j ;
int main( ) {
for(i=0 ;i < M ;i++){
for(j=0 ;j < N ;j++){
printf("%hi, ",MT[i][j]) ;}
return 0 ;}
```